Zamfirescu Alexandra
Grupa 341

SSI - Lab 6

1. Notiuni Introductive

RNG = Random Number Generator

PRNG = Pseudo RNG - biti aleatori; input de dimensiuni mici si primim ca output elemente de dimensiuni mari

TRNG = True RNG - aleator

Indiferent de seed-ul pe care-l voi da ca input, rezultatul va fi mereu 0.

- **Candidate 1**: Va returna mereu 0 (xoreaza seed-ul la infinit)
- **Candidate 2**: Va returna mereu 0 (0 + 0 / 2 = 0)
- **Candidate 3**: Doar shifteaza 2 biti la dreapta, dar tot va returna 0 => nu produce o secventa, ci doar un singur rezultat modificat => NU este un generator

2. Secrets.py

- **Generarea unei parole sigure**

```
import secrets
import string

def generate_secure_password(length):
    if length < 10:
        raise ValueError("Password length should be at least 10 characters.")

    characters = (
        string.ascii_letters +  # a-zA-Z
        string.digits +  # 0-9
        string.punctuation  # !"#$%&'()*+,-./:;<=>?@[\]^_`{|}~
    )

    while True:
```

```
        password = ''.join(secrets.choice(characters) for _ in range(length))
        if (any(c.islower() for c in password) and
            any(c.isupper() for c in password) and
            any(c.isdigit() for c in password) and
            any(c in string.punctuation for c in password)):
            return password

# Exemplu de utilizare
# Generarea unei parole sigure intr-o aplicatie de autentificare
```

- **Generare string URL-safe**

```
def generate_url_token(length):

   if length < 32:

       raise ValueError("Token length should be at least 32 characters.")

   return secrets.token_urlsafe(length)



# Exemplu de utilizare

# Token pentru resetarea unei parole, trimisa prin link in email
```

- **Generare token hexazecimal**

```
def generate_hex_token(length):

   if length < 32:

       raise ValueError("Token length should be at least 32 characters.")

   return secrets.token_hex(length)



# Exemplu de utilizare

# Token pentru autentificare API intre microservicii
```

- **Verificare sigura a doua secvente (anti timing attack)**

```
def compare_tokens(token1, token2):

   return secrets.compare_digest(token1, token2)
```

- **Generare cheie binara (pt criptare mesaj)**

```
def generate_binary_key(length=100):

   return secrets.token_bytes(length)
```

- **Stocare parole securizate**

```
import bcrypt

def hash_password(password: str) -> bytes:

   salt = bcrypt.gensalt()

   hashed = bcrypt.hashpw(password.encode('utf-8'), salt)

   return hashed



# Alegere bcrypt:

# Ofera protectie impotriva brute-force si este standardizat dpdv industrial
```

- **TESTARE**

```
(base) alexandrazamfirescu@Alexandras-MacBook-Air SSI % python3 cod.py
Parola generata: [q.LBoR]q8z7


Token URL generat: LrdXhAgacOQYE4qyp1rSEt1b5BfaZCaWlvL2K5ph6MA


Token hex generat: bd055cf0defa59e3f8da7a8f971957ca1e9ff9f17a19bcc2b583dc76d0efc3b3


Tokenurile sunt egale: False


Cheie binara generata: b'V6\x8b[\x9e\xfbS7r\xd60\xb7\xdd\xe2OX\xf9F\x16L\xa6\xbf_3\x0e\x0b\x06j0\xa2\xcf\x0c\xf0\x91\x96\xea\x16\x04r\x87\xfb\xb6\xf2\xc1\xa9\xaa\x04\xb8\xde\xf30l#
\xc3\x9f\x1f\xe2.\xee\xe4\x0cV\xcd\xdd\xd5\x1e\xb9jI]\xdc\xbdDl|\xa0,\xee\xb7\xd3\x01\xfc\x13\xa8c\x96\x00\n\x9c\xfe{\xca`\xbc\t>K\xd9\xfcj'


Parola hashuita: b'$2b$12$Re7VTBO1bxPy44S2Q/evcOTuXH61nVMJzwiy9UmALU2ZskovpGhMu'
```

3. CVE, CWE, CAPEC

a. In prima secventa de cod, seed-ul este constant, iar daca cineva il obtine, poate recrea intreaga secventa de valori generate.
In a doua secventa de cod, seed-ul bazat pe userID este previzibil, mai ales daca acesta este un numar secvential sau usor de ghicit. Functia **srand($userID)** seteaza seed-ul intr-un mod determinist, ceea ce inseamna ca aceeasi valoare de intrare va produce aceeasi valoare de iesire.
Daca un atacator cunoaste userID-ul, poate prezice SessionID-ul, facilitand atacuri de tipul Session Hijacking.

b. Problema semnalat se incadreaza in CWE-338: "Use of Predictable Seed in Pseudo-Random Number Generator (PRNG)".
Daca acelasi seed este folosit constant, spatiul posibil de seed-uri devine extrem de mic, permitand unui atacator sa incerce toate valorile posibile si sa regenereze sesiuni sau ID-uri. Astfel, valorile generate devin complet deterministe, pierzandu-si caracterul aleatoriu. CWE ID corespunzator pentru aceasta problema este CWE-330: "Use of Insufficiently Random Values".

c. In contextul atacurilor asupra PRNG-urilor, utilizarea unui seed previzibil sau a unui spatiu mic de seed-uri reprezinta o vulnerabilitate comuna. De exemplu, vulnerabilitatea CVE-2024-10026 a aratat ca utilizarea unui algoritm de hashing slab si a unor seed-uri de dimensiuni mici in gVisor de la Google a permis unui atacator să calculeze o adresa IP locala si un identificator per-boot, facilitand urmarirea dispozitivului in anumite circumstante. Acest lucru subliniaza importanta folosirii unor seed-uri mari si impredictibile.

d. Referitor la CWE:

- CWE-329 – "Not Using a Random Salt" apare atunci cand sarea utilizata in operatiuni criptografice nu este generata cu ajutorul unui PRNG adecvat, ceea ce duce la valori predictibile.

In ceea ce priveste CVE:

- CVE-2008-0166 (Debian OpenSSL PRNG Bug): O modificare eronata in codul Debian al OpenSSL a eliminat o sursa importanta de entropie, ceea ce a dus la generarea previzibila a valorilor si a afectat securitatea cheilor criptografice.

e. In acest an, au fost inregistrate aproximativ patru vulnerabilitatii CVE legate de PRNG, si anume **CVE-2023-25690**, **CVE-2023-28474**, CVE-2023-29974 si **CVE-2023-30541**.